

Para entregar la tarea, se deben seguir los siguientes pasos:

1. Subir el código del sitio web a un repositorio de GitHub.

<https://github.com/jaquelineFigueroa23/desarrollo-web-API-REST.git>

2. Descripción de la API.

La Users API es una aplicación desarrollada con Node.js y Express que implementa un servicio RESTful para la gestión de usuarios.

Su objetivo principal es permitir operaciones CRUD (crear, leer, actualizar y eliminar) de forma estructurada, validando los datos ingresados y respondiendo con códigos HTTP apropiados.

La API fue diseñada siguiendo los principios de la arquitectura REST, utilizando los métodos HTTP estándar:

GET para obtener información de los usuarios.

POST para crear nuevos registros.

PUT para actualizar información existente.

DELETE para eliminar usuarios.

Cada operación devuelve una respuesta en formato JSON, lo que facilita su integración con aplicaciones web o móviles que consuman los datos.

En esta versión, la información se gestiona en memoria mediante un arreglo de objetos dentro del servidor, simulando una base de datos. Esto permite demostrar la lógica de validación y manejo de errores sin necesidad de un motor de base de datos externo.

Entre las validaciones más importantes implementadas se encuentran:

El DPI debe tener exactamente 13 dígitos numéricos y no puede repetirse.

El email debe tener un formato válido y ser único.

La contraseña debe tener al menos 8 caracteres, incluyendo una mayúscula, un número y un símbolo especial.

Si se intenta cambiar el DPI o el email en una actualización (PUT), el sistema valida que el nuevo valor no esté registrado en otro usuario.

Los listados (GET) permiten aplicar filtros opcionales (name, email) y paginación (limit, offset).

La API devuelve respuestas con códigos HTTP descriptivos:

200 OK → solicitud exitosa.

201 Created → registro creado correctamente.

204 No Content → eliminación exitosa.

400 Bad Request → datos inválidos o formato incorrecto.

404 Not Found → usuario no encontrado.

409 Conflict → conflicto de datos (DPI o email duplicado).

El proyecto fue desplegado en Render para demostrar su funcionamiento en un entorno real y puede ser probado a través de herramientas como Postman.

El código fuente se encuentra disponible en GitHub, acompañado de instrucciones para ejecutarlo localmente y ejemplos de solicitudes y respuestas.

3. Instrucciones para ejecutar la API localmente.

clonar el repo

git clone <https://github.com/jaquelineFigueroa23/desarrollo-web-API-REST>

cd desarrollo-web-API-REST

instalar dependencias

npm install

modo desarrollo

npm run dev

la API queda en <http://localhost:3000>

4. URL de la API desplegada en Render.

<https://desarrollo-web-api-rest.onrender.com/>

5. Descripción de los endpoints con ejemplos de solicitudes y respuestas.

Abajo se colocaron las solicitudes y respuestas en la página 2 en adelante.

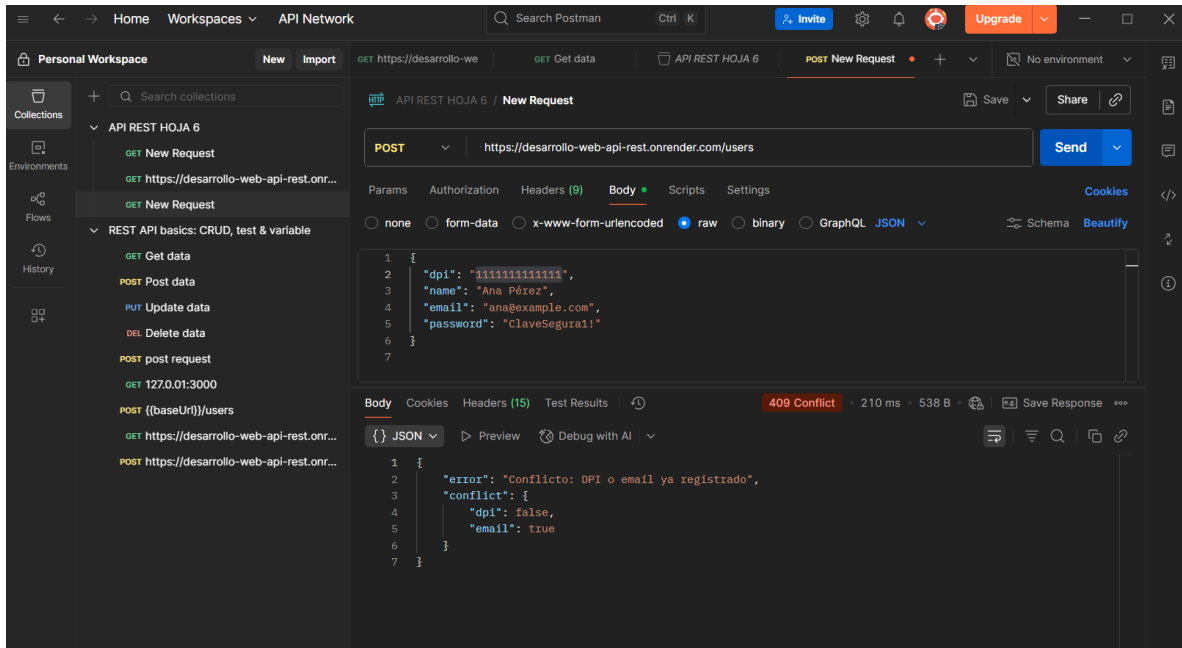
6. En la entrega de la hoja de trabajo debe subir el link del repositorio.

<https://github.com/jaquelineFigueroa23/desarrollo-web-API-REST.git>

POST

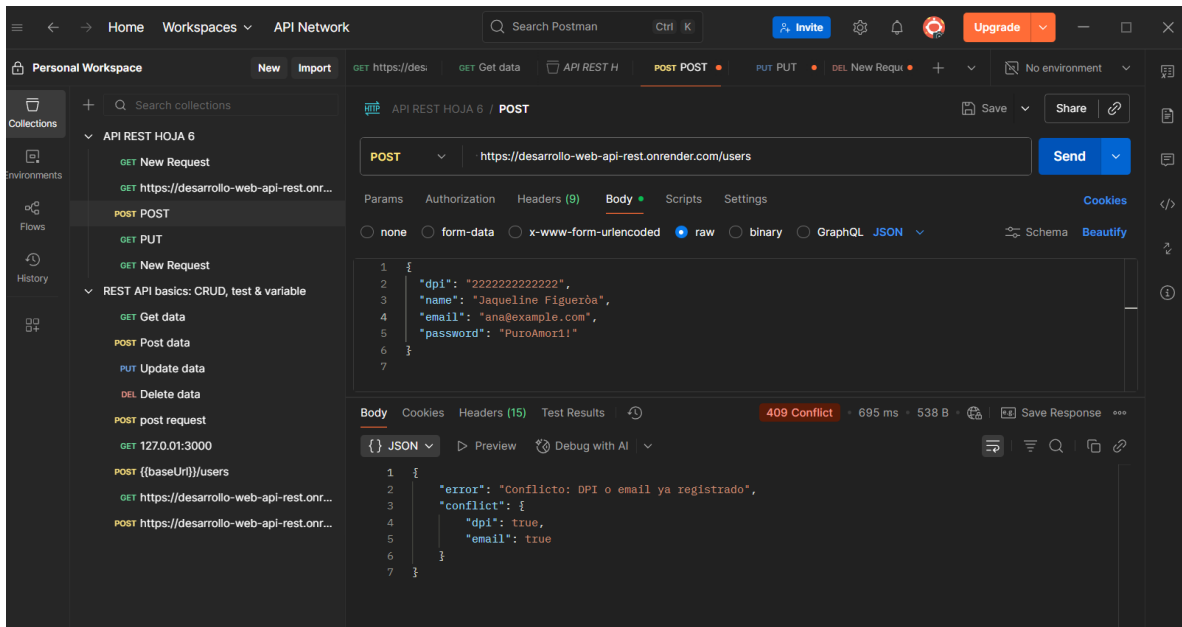
El sistema debe validar que el DPI no esté registrado previamente.

Cuando ya agregué ese dpi me muestra este error



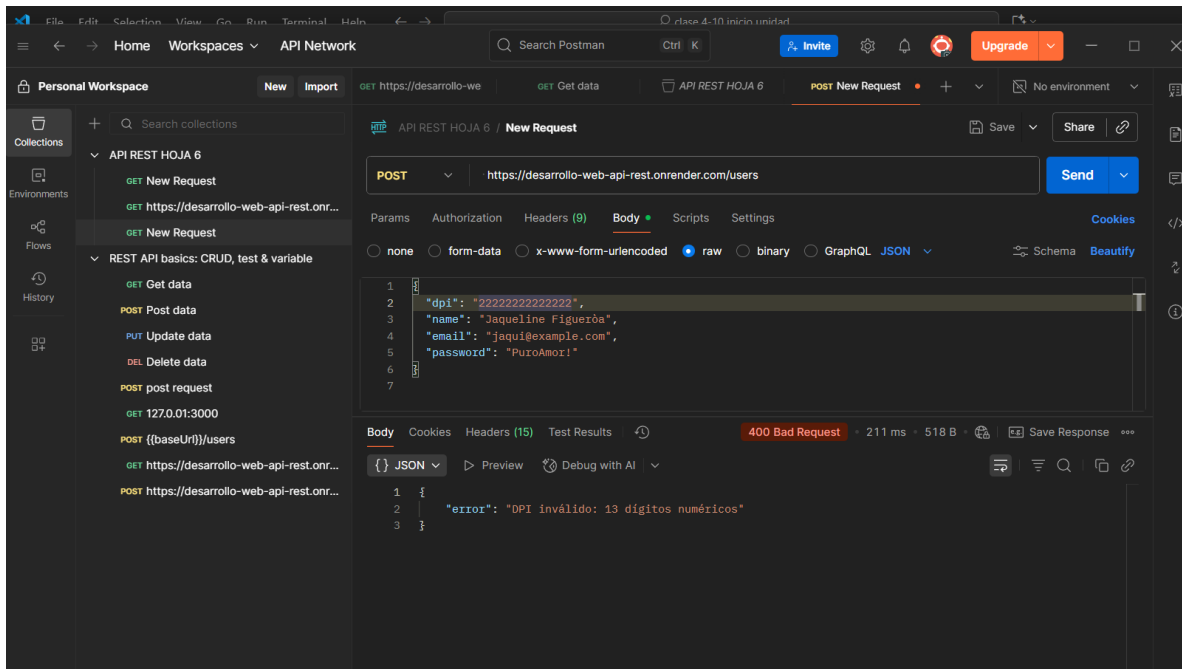
Validar que el email no se repita en otro usuario (unicidad de email). Y también valida

Si el DPI o email ya existe, devolver un error 409 Conflict.

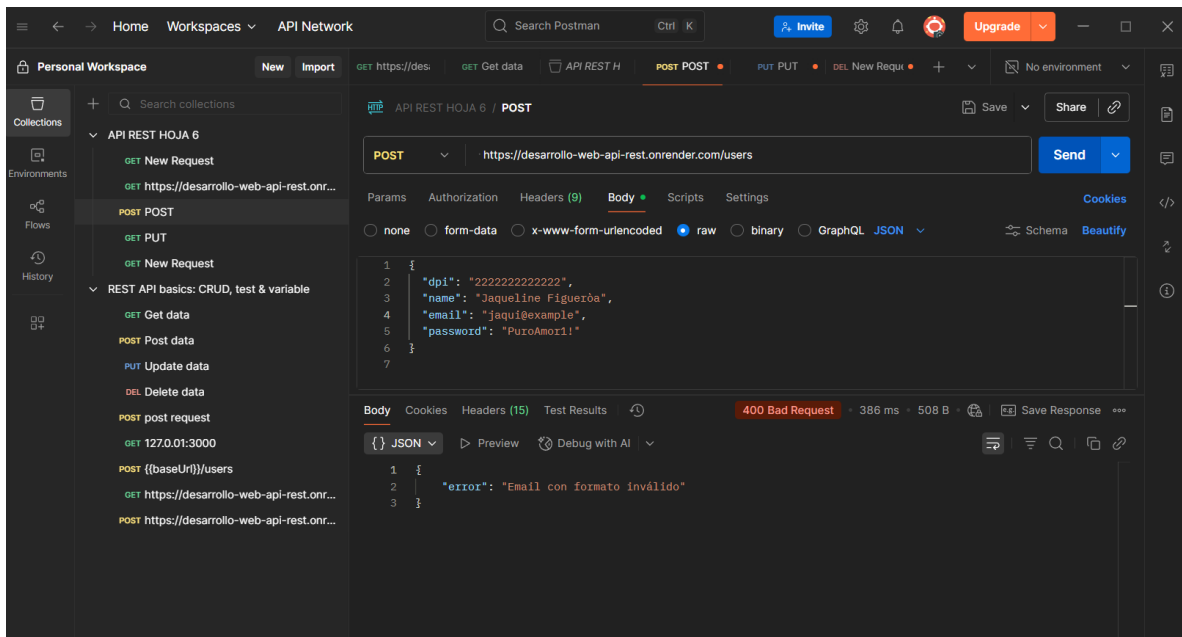


Validar que el DPI tenga exactamente **13 dígitos numéricos**.

Cuando dpi no tiene los dígitos correctos



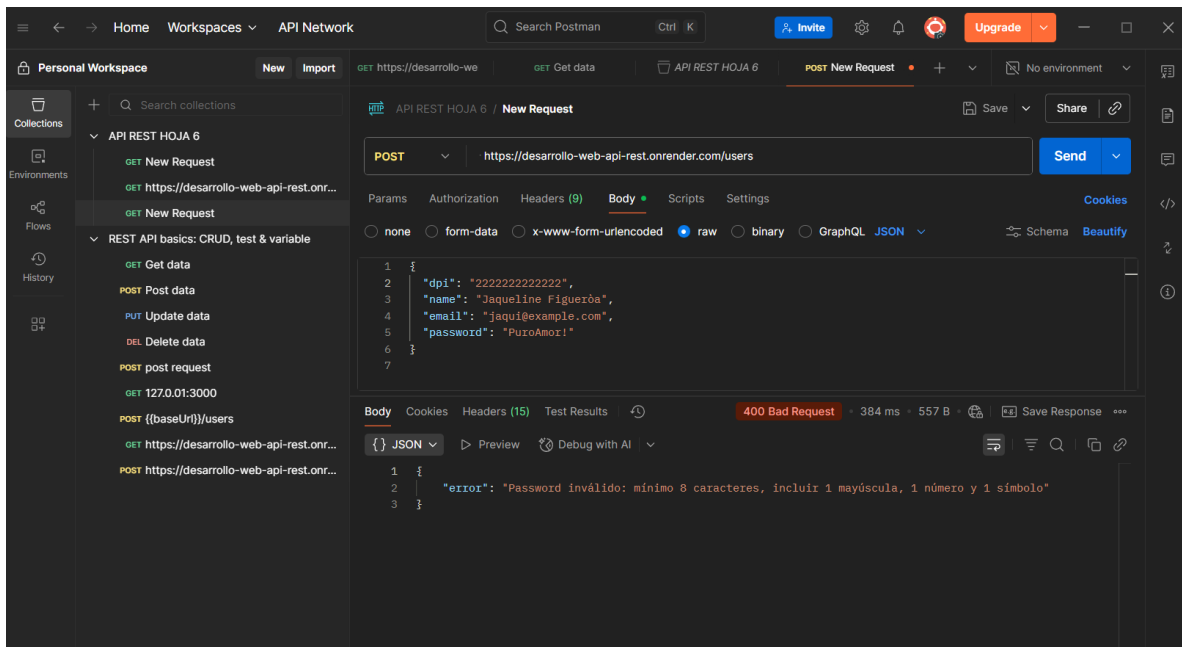
Validar que el email tenga un formato válido.



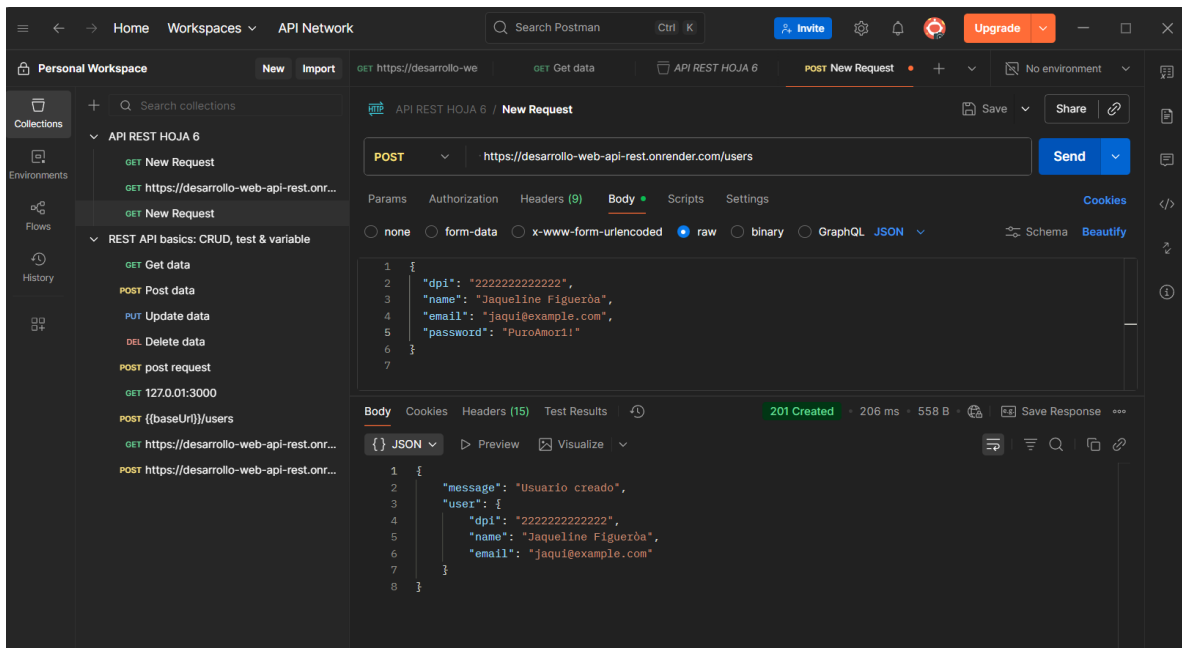
El password debe tener al menos **8 caracteres**, incluir una mayúscula, un número y un símbolo.

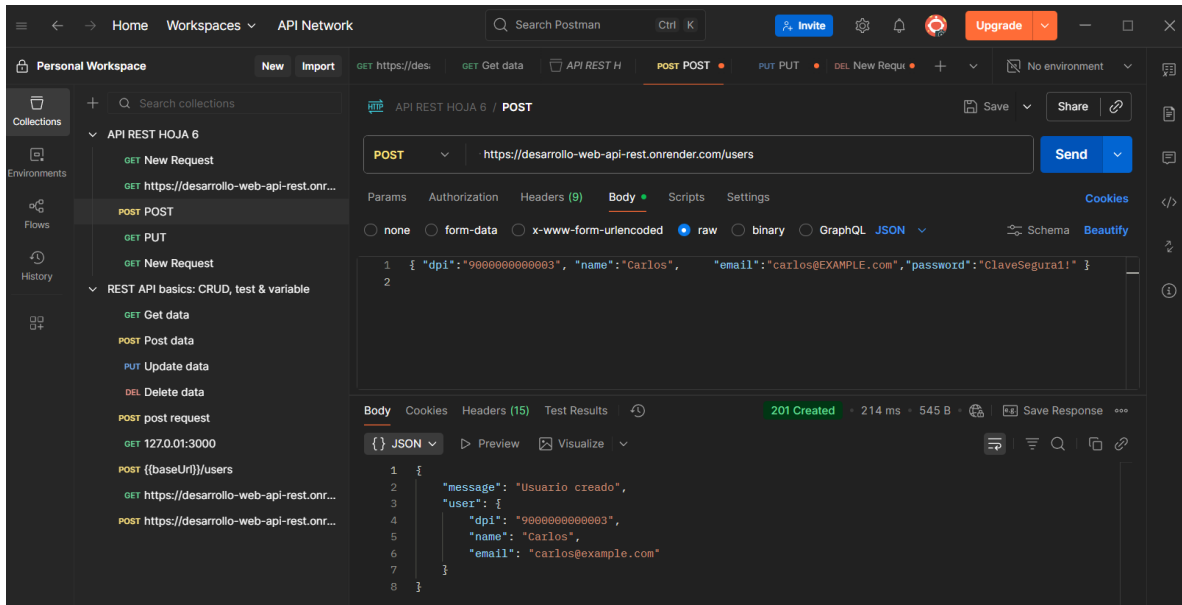
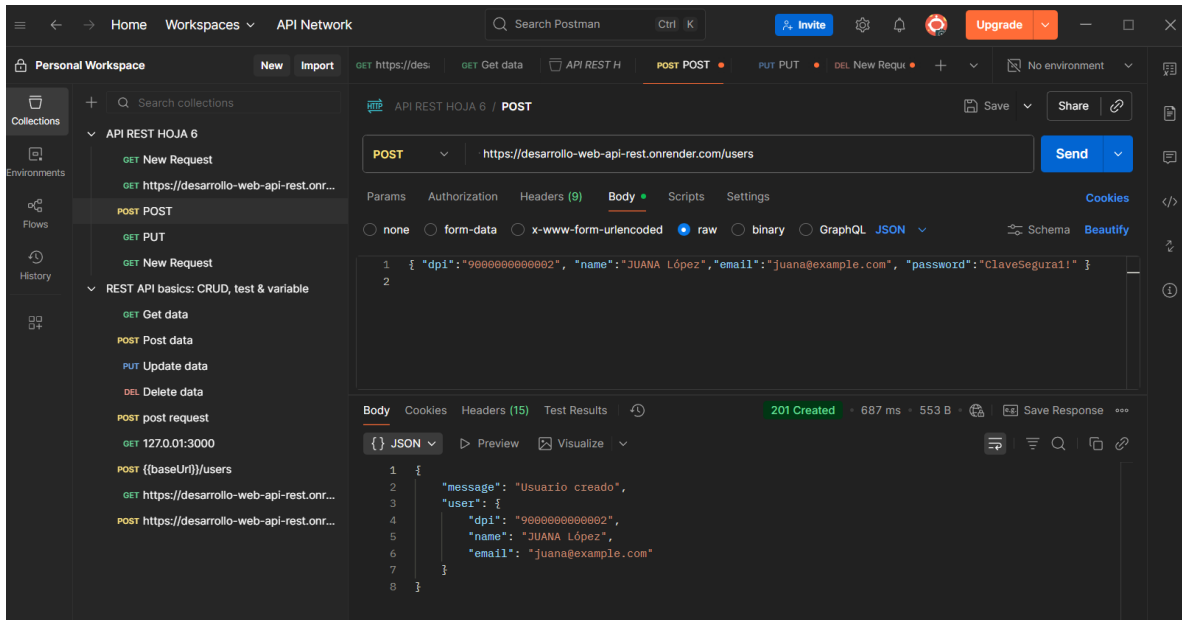
Y también se puede ver que

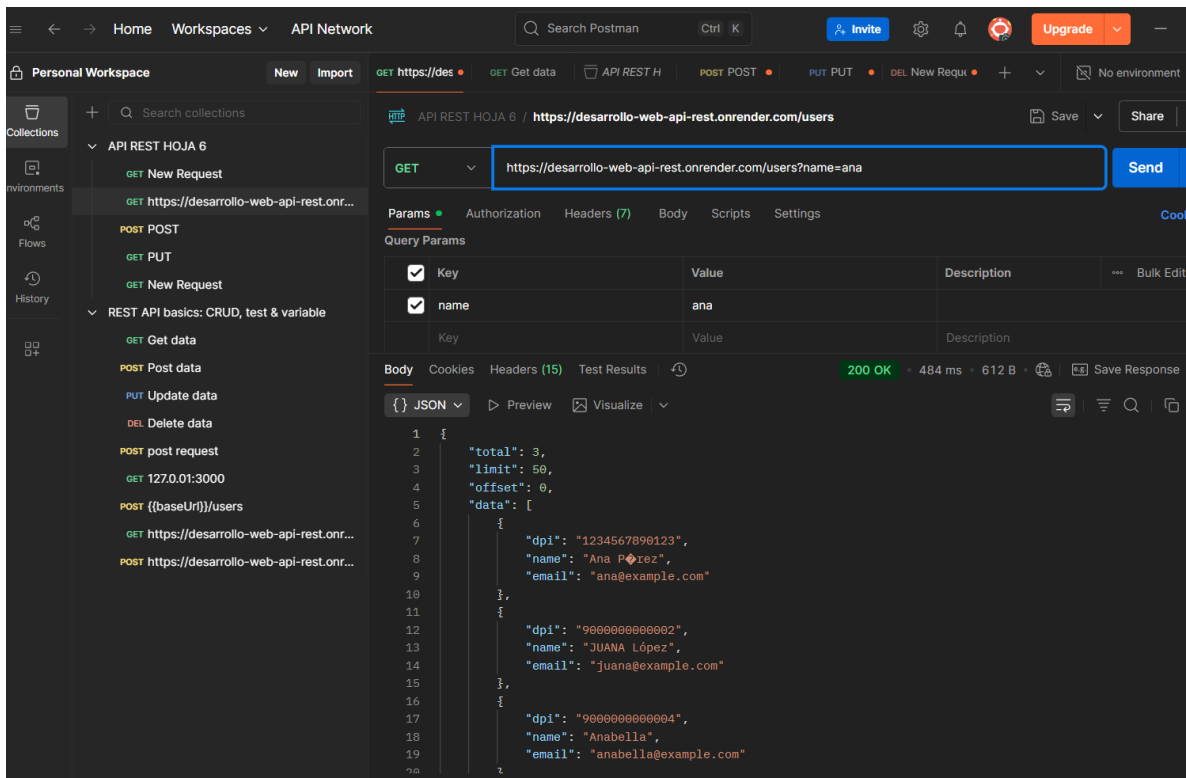
Si algún dato no es válido, devolver un error 400 Bad Request.



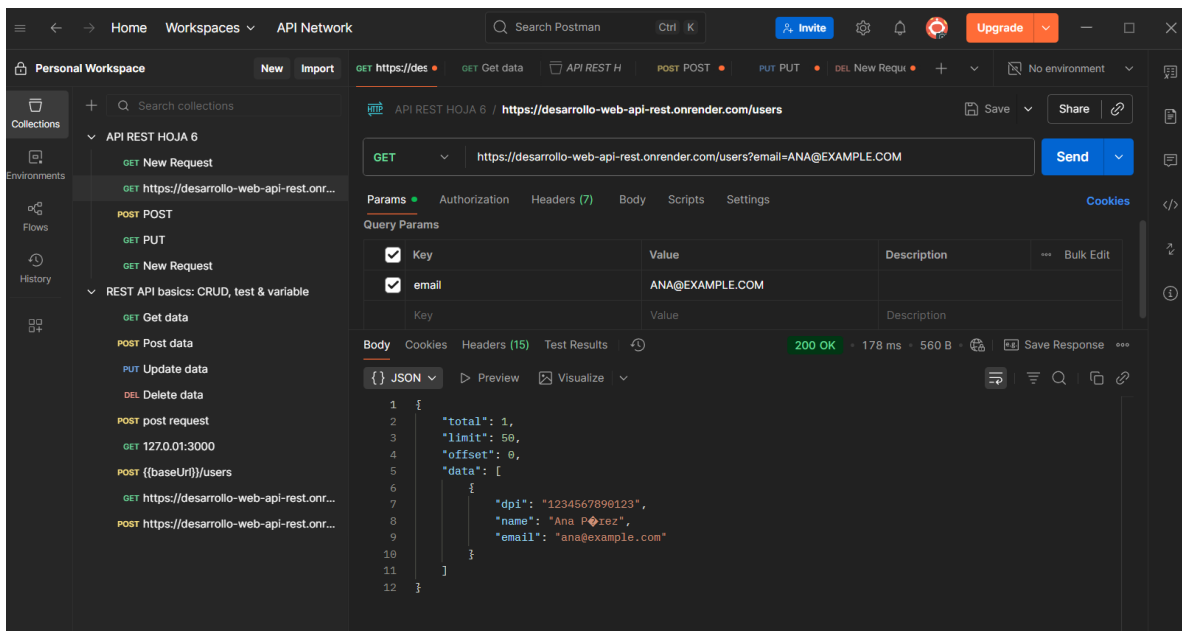
Crear correctamente usuario



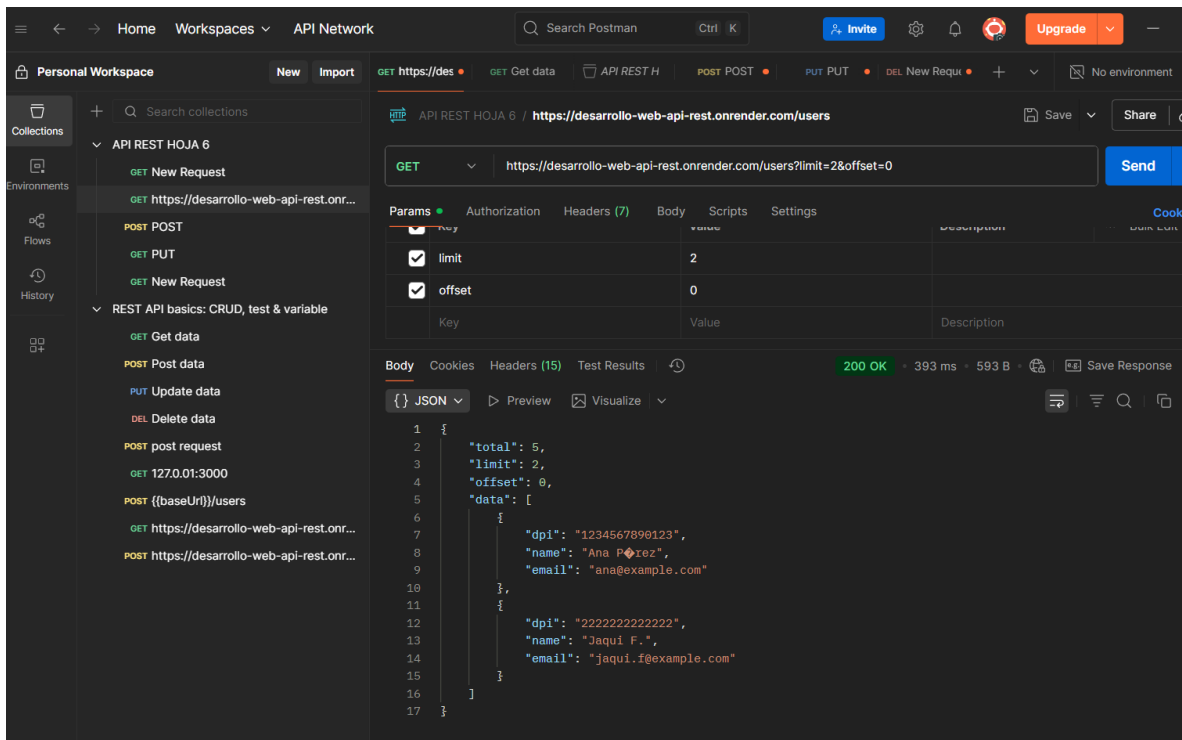




email → buscar usuario exacto por email.



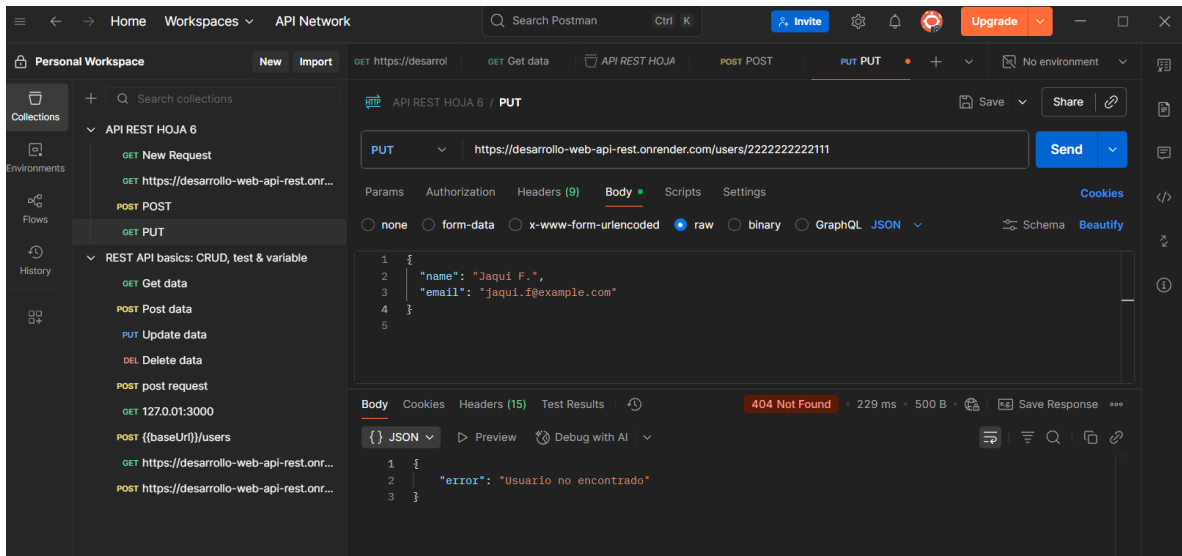
Permitir ?limit= y ?offset= para paginación.



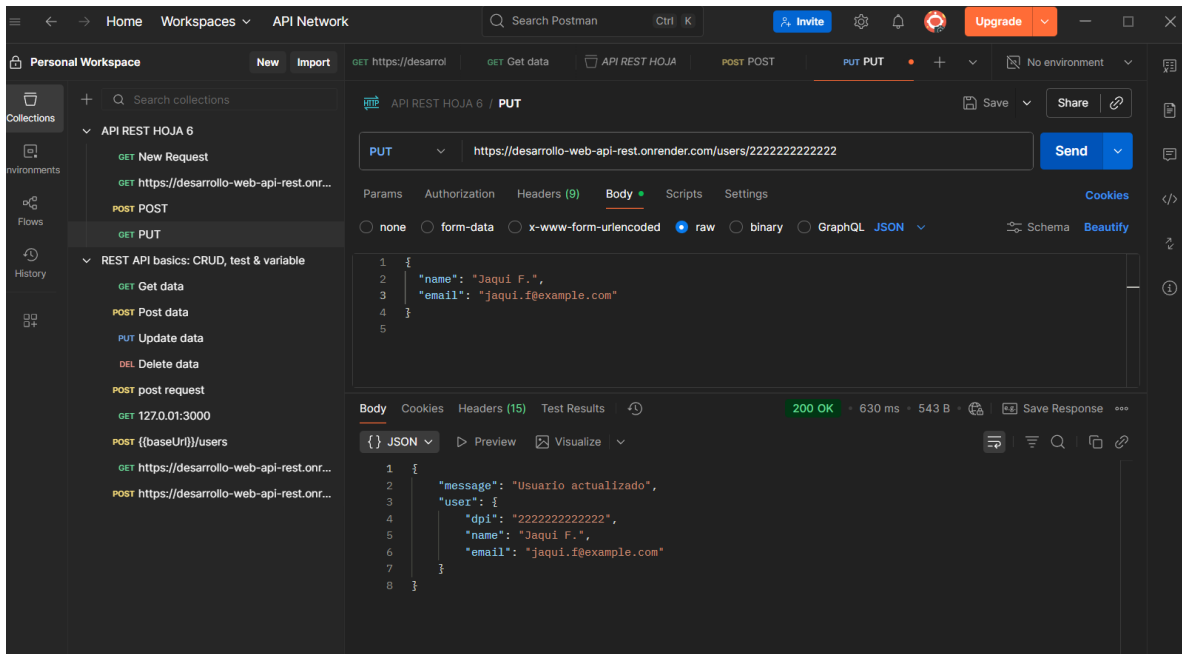
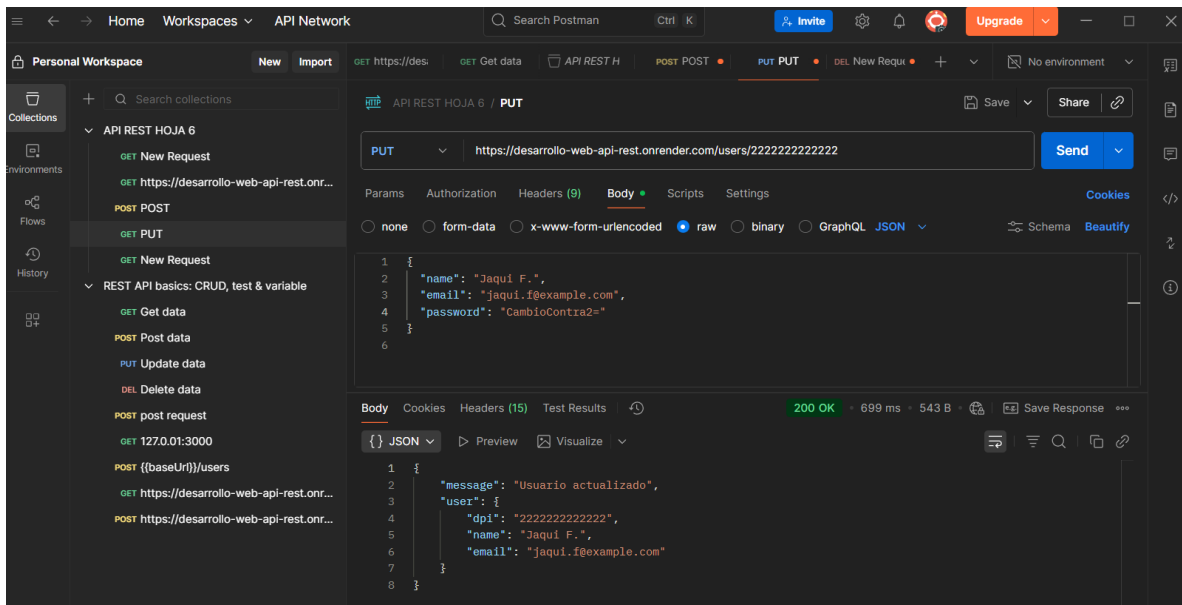
Put / actualizar

Validar que el usuario con ese DPI exista, si no, devolver 404 Not Found.

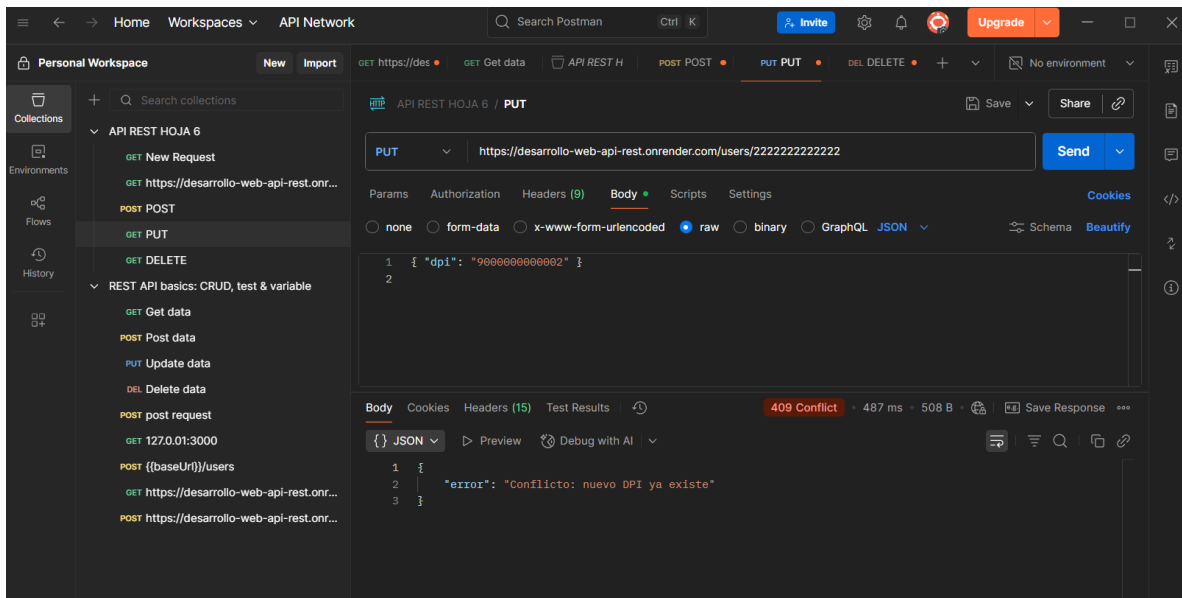
Dpi que no existe



El cuerpo de la solicitud puede incluir: name, email, password.

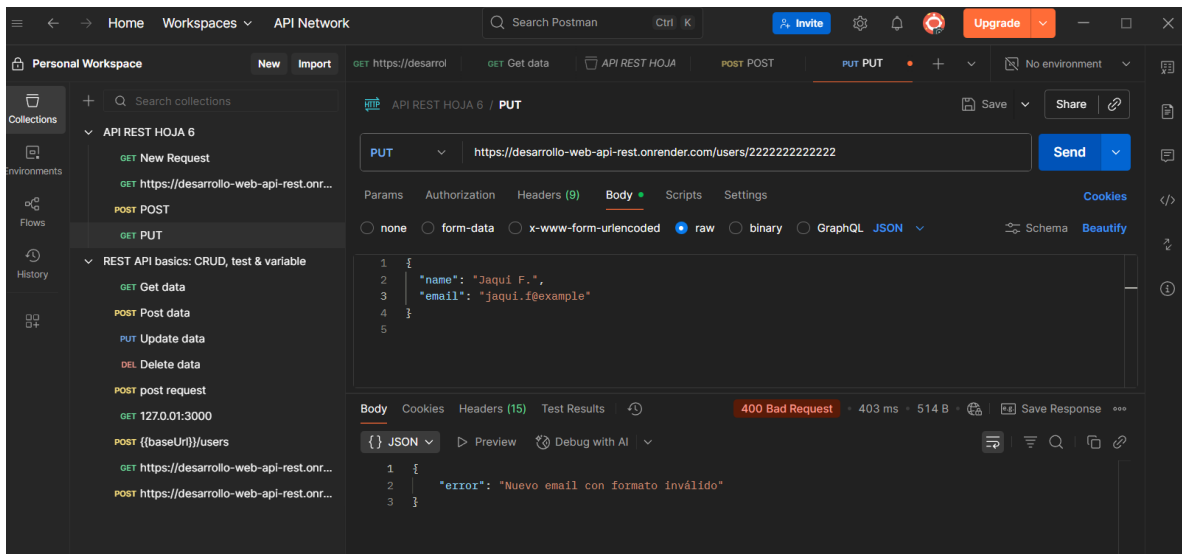


Si se intenta cambiar el DPI, validar que el nuevo no esté registrado en otro usuario.



Validar que el email no entre en conflicto con otro usuario existente.

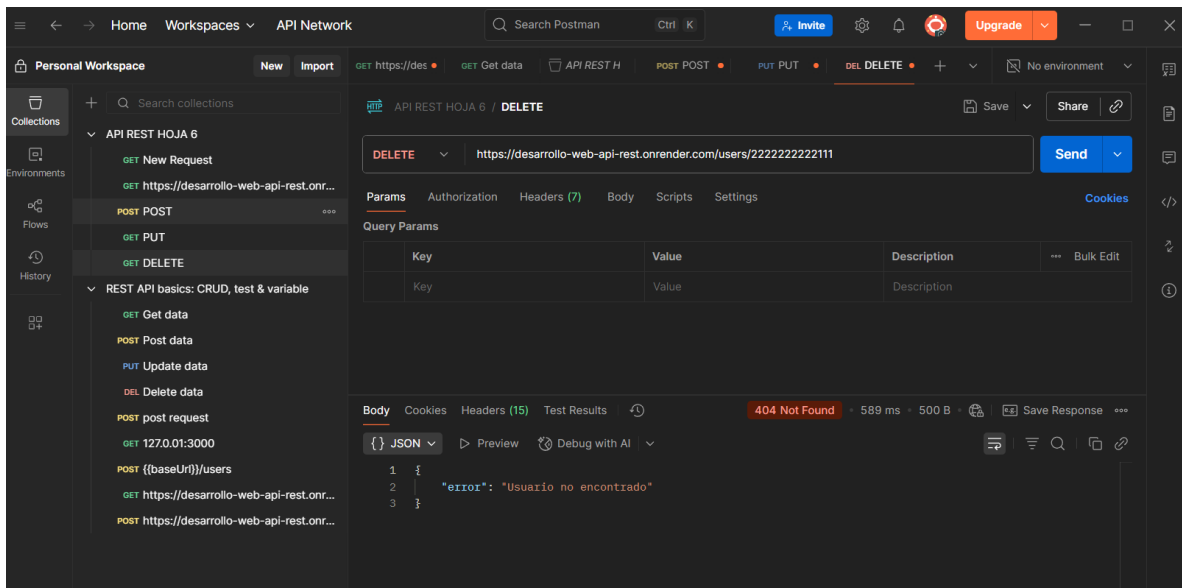
Correo invalido



Delete / eliminar

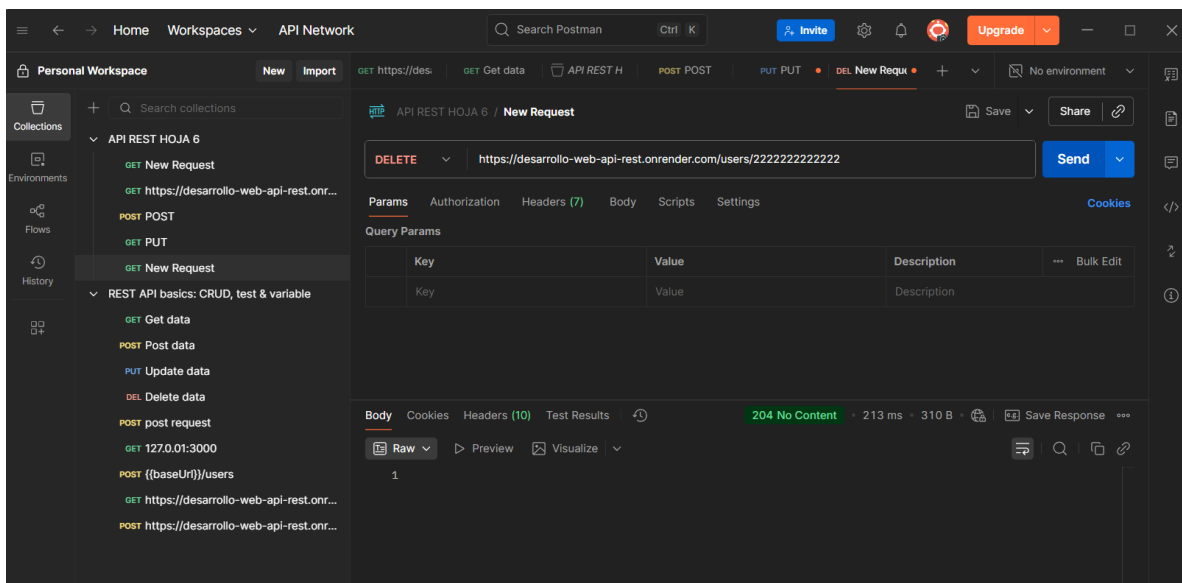
Validar que el usuario con ese DPI exista.

Si no existe, devolver 404 Not Found.

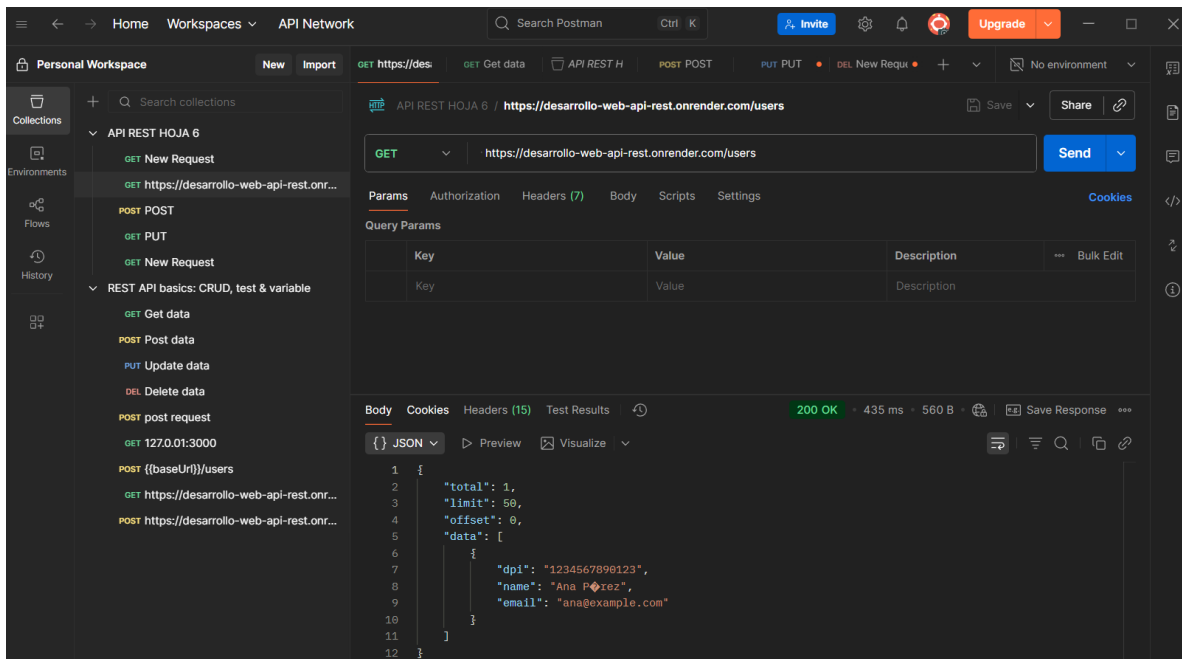


Eliminar al usuario del arreglo.

Sin cuerpo



Ya no aparece el DPI 2222222222222



Antes del delete aparecen los dos

